Dijkstra 不是贪心算法：一种从全局到局部的正确性证明

王圣宝
杭州师范大学，中国

2026 年 10 月 13 日

摘要

Dijkstra 算法历来被归类为贪心算法。本文认为，这种分类未能准确揭示算法确定最短路径的内在机制。我们给出一种直接的、从全局到局部的正确性证明。每一轮迭代中，算法实际上从所有终点尚未解决的路径中确定一条全局最短路径。该路径自然也是通向自身终点的最短路径。更根本的是，算法利用已经确定的最短路径，在每一轮中求出一条新的最短路径。本文将这一过程理解为动态规划。我们通过三步排除证明，说明如何将完整路径空间压缩为前沿候选，如何利用松弛操作排除较长路径，以及为什么最小值提取能够确定全局最小值。因此，算法的选择来自确定性的全局比较，而非乐观的局部贪心选择。我们认为，与传统贪心分类相比，动态规划与确定性路径排除更准确地揭示了 Dijkstra 算法的本质。

1 引言

Dijkstra 于 1959 年提出的最短路径算法 [1]，是计算机科学中的经典算法。该算法具有重要的实际用途，例如地图导航中的路径规划，以及通信网络中的路由计算 [2]。与此同时，它长期被视为典型的贪心算法。经典教材普遍采用这一分类 [3, 4, 5]。Kleinberg 和 Tardos 使用 *myopically*（短视地）描述逐步进行的贪心决策。Cormen 等人认为，Dijkstra 每轮选择暂定距离最小的未解决顶点，因此采用了贪心策略；随后再证明所选顶点的距离已经达到最终最优值。Sedgewick 和 Wayne 也采用选取最小距离与松弛相结合的描述，普林斯顿大学的相关课程同样将最短路径问题列入贪心算法的教学内容 [2, 6]。Roughgarden 给出了一个特别清晰的例子：先选择一个边界上的最小候选，再证明这一候选对于所选终点是最优的 [7]。

上述证明是严谨的，而且也利用了路径首次跨越已解决集合边界的性质。不过，它们主要证明*被选中终点*的最短距离已经确定。证明通常先给出选择规则，再说明该选择为何正确。这种组织方式容易使人将算法理解为：先执行局部贪心选择，再证明它是安全的。但它没有直接揭示一个更强的事实：被选中的候选实际上是*所有终点尚未解决的路径*中的全局最小值。

关于 Dijkstra 算法，也出现过不同于常规贪心叙述的解释。借助动态规划中熟知的最优子结构思想 [8, 9]，Sniedovich 强调 Dijkstra 的动态规划性质；Misra 则从不动点计算出发推导该算法 [10, 11]。不过，Sniedovich 仍沿用对算法“贪心选择规则”的描述；Lew 在讨论动态规划与贪心算法的联系时，也继续将 Dijkstra 作为贪心算法 [12]。这些工作没有从所有未解决终点路径的全局最小化出发，依据贪心策略的基本含义否定这一分类，更没有给出本文的全局到局部三步排除证明。

本文从这个全局路径空间出发，证明每一轮迭代确实找到了其中的绝对最短路径。由此，该路径终点的最短路径性质自然成立。为说明这一点，我们提出三步排除证明：首先将搜索限制在

前沿；其次利用最优子结构与松弛操作，为每个前沿顶点保留最佳候选；最后从剩余候选中确定全局最小值。这三种排除恰好对应 Dijkstra 算法本身的主要操作。

这种解释改变了传统论证的逻辑方向。算法并非相信某个局部上看似有利的选择，随后才获得全局结论；相反，确定性的排除过程先确立全局最小值，再将其终点标记为已解决。每一轮都利用既有最短路径推出一条新的最短路径。因此，本文主张 Dijkstra 本质上是一种动态规划算法，将其归类为贪心算法并不准确。这一判断针对的是最小值选择的数学依据，而非否定已有正确性证明的有效性。

2 预备知识

设 $G=(V,E,w)$ 是一个有限的带权有向图，源点为 $s\in V$。为使排除论证更清晰，本文统一假定各条边的权重严格为正：

$$w(e)>0 \qquad (e\in E).$$

严格正权保证路径每次继续延伸都会变长。Dijkstra 算法同样适用于零权边：此时路径延伸后虽可能不变长，但不会变短，前沿候选仍能达到当前全局最小值。最短路径上可能出现距离相等的顶点，因此不再保证其顶点严格按照前缀顺序解决。负权边则可能破坏这一论证。本文仍采用严格正权假设，以简化比较。

本文的路径指顶点序列 $P=(s=v_0,v_1,\dots,v_k=v)$，起点固定为源点 s，$v_1,\dots,v_{k-1}$ 为中间途经顶点，v 为终点。以下只考虑简单路径，即没有重复顶点的路径。在正权条件下，删除游走中的回路会严格缩短路径，因此这种限定不会遗漏任何最短路径。记路径的边权和为 $w(P)$；对于从 s 可达的顶点 v，用 $\delta(s,v)$ 表示最短路径距离。不可达顶点不参与下面的有限距离最小化；若不存在可达的未解决顶点，算法即终止。

2.1 三个顶点集合

Dijkstra 在其 1959 年原始论文的 Problem 2 中，已经将顶点分为三类 [1]：集合 A 为最短路径已经确定的顶点；集合 B 为与 A 直接相连的其余顶点，下一个顶点从 B 中选出；集合 C 包含所有剩余顶点。本文沿用这一三区域结构，但采用更便于证明的记号。

在第 t 轮迭代中，设 $\mathcal{S}_t$ 为已解决集合，其中各顶点满足

$$d(x)=\delta(s,x) \qquad (x\in\mathcal{S}_t),$$

并设 $\mathcal{U}_t=V\setminus\mathcal{S}_t$ 为未解决集合。初始时，$\mathcal{S}_t=\{s\}$，且 $d(s)=0$。在 $\mathcal{U}_t$ 内进一步定义

$$\mathcal{F}_t=\{f\in\mathcal{U}_t:\exists x\in\mathcal{S}_t \text{ 使得} (x,f)\in E\}$$

为前沿集合（frontier）；设 $\mathcal{I}_t=\mathcal{U}_t\setminus\mathcal{F}_t$ 为不可见集合（invisible set）。因而

$$V=\mathcal{S}_t\,\dot\cup\,\mathcal{U}_t, \qquad \mathcal{U}_t=\mathcal{F}_t\,\dot\cup\,\mathcal{I}_t.$$

这里“不可见”仅表示相关顶点尚未通过一条从已解决区域向外的边直接暴露。图 1 用本文符号展示了 Dijkstra 原文已有的三个区域。对于有向图，前沿由从已解决集合指向外部的边确定。

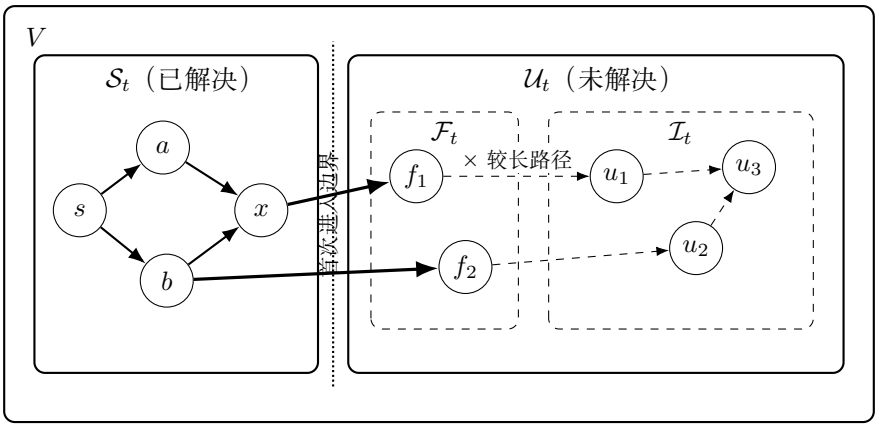


图 1: 第 t 轮迭代中的三个顶点区域。任何从源点 s 到未解决顶点的路径，都必须首先进入前沿集合 $\mathcal{F}_t$。由于边权严格为正，从该处继续延伸必定增加路径长度。

2.2 一个启发性的起步

源点 s 的最短距离显然为 $\delta(s,s)=0$，因为存在长度为零的平凡路径。这只是已知的初始条件，并非一次需要求解的最短路径搜索。Dijkstra 算法的第一次实际求解，是确定到某个其他顶点的最短路径。根据最小暂定距离选择规则，很容易看出：该顶点就是 s 指向其他顶点的最小权重出边所指向的顶点。若存在从 s 可达的其他顶点，记

$$m = \min_{\substack{(s,v)\in E \\ v\neq s}} w(s,v).$$

这样的单边路径长度为 m。任何从 s 到其他顶点、至少包含两条边的简单路径，其第一条边的权重不小于 m，之后还要增加正权边。所以，最小权重出边对应的路径不仅是到自身终点的最短路径，而且是所有从 s 出发、终点为其他顶点的路径中的绝对最短路径。

其道理非常简单：路径增加边，就会变长。这便引出了本文的核心问题：*后续每一轮是否同样能从所有终点尚未解决的路径中确定一条绝对最短路径？*下面的全局到局部证明给出肯定回答。

3 基于三步排除的全局到局部证明

3.1 全局路径最小化

对于每个 $u\in\mathcal{U}_t$，记 $\mathcal{P}(s,u)$ 为从 s 到 u 的路径集合。定义第 t 轮的全局待解路径空间为

$$\mathcal{P}(\mathcal{U}_t) = \bigcup_{u\in\mathcal{U}_t} \mathcal{P}(s,u).$$

当前轮的目标是确定

$$P_t^* \in \arg\min_{P\in\mathcal{P}(\mathcal{U}_t)} w(P).$$

这里是*在所有终点尚未解决的路径中取最小值*，而不是先假定各未解决顶点的最短路径已经求出，再对它们进行比较。

从全局最小值推出局部最短路径。 设 P_t^* 的终点为 u^*。因为

$$\mathcal{P}(s,u^*) \subseteq \mathcal{P}(\mathcal{U}_t),$$

所以任意从 s 到 u^* 的路径 Q 均满足

$$w(P_t^*) \le w(Q).$$

因此，一条路径若在全局待解路径空间中取得最小值，它必然也在自身终点对应的路径集合中取得最小值。每次找到一条全局最短路径，就解决了一个新的顶点。剩下的问题是：Dijkstra 如何在不枚举所有路径的情况下确定这个全局最小值?

排除原则。 为了寻找最短路径，可以排除那些已经确定比其他有效候选更长的路径。若 P_1 仍然有效且 $w(P_1) < w(P_2)$，则 P_2 不可能取得当前最小值。即使边权严格为正，最短路径也未必唯一。本文所说的 “一条全局最短路径”，指任意一个最小值实现者；如果存在多个等长的最优候选（路径终点相同或者不同），任取其中一条即可，最短距离不受影响。

这里的排除只针对当前这一轮。一条路径在进入第一个未解决顶点后继续延伸，可能与当前全局最小值无关；但当该顶点被解决后，其后续边又可能成为重要候选。下面的证明利用这一区别缩小当前搜索空间，并不会丢失未来可能需要的最短路径。

3.2 三步排除正确性证明

固定第 t 轮，假设存在至少一个可达的前沿顶点。对于 $x \in \mathcal{S}_t$，最短距离 $d(x) = \delta(s,x)$ 已经确定。下面证明下一次最小值提取能够得到 $\mathcal{P}(\mathcal{U}_t)$ 中的全局最小值。

第一步：前沿排除。 任取

$$P = (s = v_0, v_1, \dots, v_k = u) \in \mathcal{P}(\mathcal{U}_t).$$

设 v_j 为 P 上第一个未解决顶点。由于 $v_0 = s \in \mathcal{S}_t$，其前驱 v_{j-1} 必然属于 $\mathcal{S}_t$。又因为 $(v_{j-1}, v_j) \in E$，按照前沿集合的定义，有 $v_j \in \mathcal{F}_t$。以 v_j 为终点的前缀 $P^{\text{first}} = (v_0, \dots, v_j)$ 同样属于 $\mathcal{P}(\mathcal{U}_t)$。如果 P 还要越过 v_j，它至少会增加一条正权边，因此

$$w(P^{\text{first}}) < w(P).$$

可见，包含两个及以上未解决顶点的路径不可能是当前全局最短路径，因为它截至第一个前沿顶点的前缀已经更短。于是本轮只需考虑

$$s \rightsquigarrow x \to f, \qquad x \in \mathcal{S}_t, \quad f \in \mathcal{F}_t,$$

且所有中间顶点属于已解决集合，唯一的未解决顶点是终点 f。这说明为什么当前只需考虑前沿顶点，无须进一步探索深层未解决路径。这里的排除是相对于该路径自身更短的首次跨界前缀而言；被暂时排除的路径，并不需要长于所有其他保留的前沿候选。

第二步：松弛排除。 固定 $f \in \mathcal{F}_t$。第一步之后保留的每条候选路径均有形式

$$P = Q(s,x) \circ (x,f), \qquad x \in \mathcal{S}_t.$$

首先考察到 x 的前缀。由于 x 已经解决，任意前缀 $Q(s,x)$ 的长度不可能小于 $d(x) = \delta(s,x)$。若它更长，则用已知最短前缀替换后可以构成一条更短的到 f 候选路径。所以，对每条跨界边 (x,f)，只需保留候选值

$$d(x) + w(x,f).$$

这就是最优子结构所发挥的作用：松弛操作直接利用已经确定的最短距离，而不必枚举此前通向 x 的各种路径。

同一个前沿顶点 f 可能具有多个已解决前驱。它们产生的候选值中，最小者为

$$D_t(f) = \min_{\substack{x \in \mathcal{S}_t \\ (x,f) \in E}} \{d(x) + w(x,f)\}.$$

其他到 f 的较长候选均可排除；如果出现等长，只需保留一个代表。因此，第二步同时包含“最优前缀替换”和“同终点最小化”，二者都由松弛操作体现。经过这一步，当前搜索由前沿值集合

$$\{D_t(f) : f \in \mathcal{F}_t\}$$

表示。

第三步：全局最小值提取。 选取

$$f^* \in \arg\min_{f \in \mathcal{F}_t} D_t(f), \qquad D^* = D_t(f^*).$$

下面证明，D^* 不仅是前沿候选的最小值，而且是整个 $\mathcal{P}(\mathcal{U}_t)$ 的最小值。

任取 $P \in \mathcal{P}(\mathcal{U}_t)$，设 f 是其第一个未解决顶点，x 是 f 在路径上的前驱，L_x 是从 s 到 x 的前缀长度。由于 $x \in \mathcal{S}_t$，有 $d(x) \le L_x$。又因为 $D_t(f)$ 是所有从已解决区域跨界到 f 的候选中的最小值，故

$$D_t(f) \le d(x) + w(x,f) \le L_x + w(x,f).$$

路径 P 在 f 之后的剩余部分长度非负，因而 $w(P) \ge L_x + w(x,f)$。再由 f^* 的选取知 $D_t(f) \ge D_t(f^*)$。综合得到

$$\begin{aligned} w(P) &\ge L_x + w(x,f) \\ &\ge d(x) + w(x,f) \\ &\ge D_t(f) \\ &\ge D_t(f^*) = D^*. \end{aligned}$$

确实存在一条长度为 D^* 的路径：沿某个已解决前驱的最短路径，再接上相应跨界边。根据此前各轮的归纳构造，该最短前缀可以完全位于已解决集合内，因此接上指向未解决顶点的边后仍是简单路径。因此

$$D^* = \min_{P \in \mathcal{P}(\mathcal{U}_t)} w(P) = \min_{f \in \mathcal{F}_t} D_t(f).$$

该等式也说明了最小暂定距离选择的来源：前两步已在保持全局最小值不变的前提下完成候选压缩，最后只需对保留的前沿标签取最小值。

定理 1 (单轮迭代的正确性)**.** *若所有边权均为正，且至少有一个可达的未解决顶点，则任何使 $D_t(f)$ 在 $\mathcal{F}_t$ 上取得最小值的顶点 f^* 都满足*

$$D_t(f^*) = \delta(s, f^*).$$

因此，其最短路径距离可以被最终确定。

证明. 由上述不等式可知，一条达到 $D_t(f^*)$ 的路径就是 $\mathcal{P}(\mathcal{U}_t)$ 中的全局最短路径。特别地，它必然也是所有以 f^* 为终点的路径中的最短路径，因为后者是 $\mathcal{P}(\mathcal{U}_t)$ 的子集。 □

随后，已解决集合扩大，未解决集合缩小：

$$\mathcal{S}_{t+1} = \mathcal{S}_t \cup \{f^*\}, \qquad \mathcal{U}_{t+1} = \mathcal{U}_t \setminus \{f^*\}.$$

下一轮可以重复同样的论证。

3.3 与标准 Dijkstra 算法的对应

三步排除并不是额外增加的算法操作，而是对标准算法已有操作的逻辑解释，如表 1 所示。前沿限制对应第一步；松弛操作维护第二步得到的最优候选值；Extract-Min 完成第三步。

表 1: 三步排除与 Dijkstra 标准操作的对应关系。

排除步骤	标准操作	数学作用
第一步：前沿排除	前沿限制	排除越过第一个未解决顶点后继续延伸的路径。
第二步：松弛排除	暂定距离的松弛更新	使用最优前缀，并为每个前沿顶点保留最佳候选。
第三步：全局最小值	Extract-Min	从全部剩余候选中选出全局最小值。

为了更明确地说明三步排除证明与原算法流程的对应，算法 1 给出一份简洁的 Dijkstra 伪代码，并在相应操作旁标注排除步骤。初始时，源点 s 已加入已解决集合，算法立即对其出边进行松弛，因此第一次最小值提取使用的前沿值已经计算完成。在之后的每一轮中，提取之后进行的松弛，为下一次提取更新前沿候选值。所以，虽然循环内部先执行 Extract-Min 再执行松弛，但每次提取所依赖的状态，实际上已经完成了第二步。两处出边扫描还明确体现了第一步：只考察从已解决顶点指向未解决顶点的边，因此只有这些边的终点，即前沿顶点，会进入当前候选。

一旦顶点 u 被解决，$d(u) = \delta(s, u)$ 就不再改变，故扫描其出边 (u, v) 时，$d(u) + w(u, v)$ 已是该边所能提供的最佳扩展值。在标准邻接表实现中，从可达顶点出发的每条有向边至多被检查一次；若其终点尚未解决，才尝试松弛。同一个前沿顶点的暂定距离仍可能多次减小。相比之下，标准 Bellman–Ford 算法必须多轮扫描边，因为边起点的距离估计仍可能改进。这也解释了 Dijkstra 算法效率较高的重要原因。

Dijkstra 原文已经采用三个顶点集合，并反复执行候选更新与最小值选择 [1]。本文证明揭示了其背后的全局路径最小化依据，解释的是原有算法，并非另造一套算法。

Algorithm 1 以源点作为初始已解决顶点的 Dijkstra 算法

```
d(s) ← 0；对 v ≠ s，置 d(v) ← ∞
S ← {s}；Q ← V \ {s}（以 d 为键的优先队列）
for 每条 (s, v) ∈ E 且 v ∈ Q do                ▷ 第一步：仅考虑前沿终点
    d(v) ← min{d(v), w(s, v)}；更新 Q 中 v 的键值   ▷ 第二步：初始松弛
end for
while Q ≠ ∅ do
    u ← ExtractMin(Q)                          ▷ 第三步：前沿最小值
    if d(u) = ∞ then
        break
    end if
    S ← S ∪ {u}
    for 每条 (u, v) ∈ E 且 v ∈ Q do             ▷ 第一步：仅考虑前沿终点
        if d(u) + w(u, v) < d(v) then
            d(v) ← d(u) + w(u, v)；更新 Q 中 v 的键值   ▷ 第二步：松弛
        end if
    end for
end while
```

4 讨论：动态规划而非贪心选择

4.1 动态规划递推

式 (3.2) 利用已经确定的最短距离计算前沿候选；新的全局最小距离一经确定，又成为后续迭代的已解子问题。由此，算法通过已求出的最短路径逐步构造新的准确结果。记 $L_t = \min_{P\in\mathcal{P}(\mathcal{U}_t)} w(P)$。由于 $\mathcal{P}(\mathcal{U}_{t+1}) \subseteq \mathcal{P}(\mathcal{U}_t)$，有 $L_{t+1} \geq L_t$。所以，算法按最短路径长度的非降序依次确定各顶点的最短路径；即使边权严格为正，不同终点也可能具有相同的最短距离。正权条件下，任意最短路径的真源点前缀都更短，其终点必然较早解决。但顶点的发现顺序未必与解决顺序一致。

这一递推可与 Fibonacci 数列相比较：

$$F_0 = 0, \qquad F_1 = 1, \qquad F_n = F_{n-1} + F_{n-2}.$$

两者都从已解决子问题构造新的准确结果。不同的是，Fibonacci 的下一下标预先确定，而 Dijkstra 根据当前前沿决定下一终点。两者的共同点是依赖已知子问题的解，而非猜测下一问题的答案。

4.2 为什么贪心分类不正确

贪心策略按其基本含义，是在不考察整个剩余问题的情况下，直接作出当前局部上最有利的选择；Kleinberg 和 Tardos 使用 *myopic*（短视的）描述这一思想 [4]。Dijkstra 的 Extract-Min 在程序形式上确实很像这种选择。但上述证明的逻辑方向正好相反：被提取的前沿最小标签，已

经被证明等于所有终点尚未解决的路径的全局最小值。所谓局部选择，只是经过确定性排除之后，全局最小化在程序中的简洁实现；其依据不是短视决策。再结合算法基于已解最短路径的递推结构，本文将 Dijkstra 归为动态规划算法，而非贪心算法。

5 结论

我们给出了 Dijkstra 算法的一种从全局到局部的正确性证明。三个确定性的排除步骤，依次将所有未解决终点路径缩减为前沿候选，再压缩为每个前沿顶点对应的一个最优候选，最终确定当前全局最短路径。该路径终点的最短路径最优性，则直接由全局最小性推出。同一论证还说明了正边权条件、最短路径的依次求解，以及每条出边无须重复处理的原因。

算法每轮利用此前已经确定的最短路径，求出一条新的最短路径。这是一种动态规划过程，而不是推测性的局部贪心选择。由此，Dijkstra 与 Bellman–Ford 和 Floyd–Warshall 等早已普遍被理解为动态规划的经典最短路径算法，可以置于一个更加统一的理论框架下 [5]。这为主要经典最短路径算法提供了更一致的概念分类。

致谢

作者感谢 Zimo Wang 所提供的有益讨论，这些讨论使本文的核心思想更加清晰。作者同时感谢人工智能大语言模型 Gemini 和 ChatGPT 在稿件起草、翻译和编辑润色过程中提供的辅助。

参考文献

[1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959. doi:10.1007/BF01386390.

[2] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ: Addison-Wesley Professional, 2011.

[3] G. Brassard and P. Bratley, *Algorithmics: Theory and Practice*. Englewood Cliffs, NJ: Prentice Hall, 1988.

[4] J. Kleinberg and E. Tardos, *Algorithm Design*. Boston, MA: Pearson/Addison-Wesley, 2006.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.

[6] K. Wayne, “COS 423: Analysis of Algorithms—Greedy Algorithms: Shortest Path,” Princeton University, course materials. [Online]. Available: https://www.cs.princeton.edu/~wayne/cs423/.

[7] T. Roughgarden, *Algorithms Illuminated, Part 2: Graph Algorithms and Data Structures*. San Francisco, CA: Soundlikeyourself Publishing, LLC, 2018.

[8] R. Bellman, *Dynamic Programming*. Princeton, NJ: Princeton University Press, 1957.

[9] E. L. Lawler, *Combinatorial Optimization: Networks and Matroids*. New York, NY: Holt, Rinehart and Winston, 1976.

[10] M. Sniedovich, "Dijkstra's algorithm revisited: The dynamic programming connexion," *Control and Cybernetics*, vol. 35, no. 3, pp. 599–620, 2006.

[11] J. Misra, "A walk over the shortest path: Dijkstra's algorithm viewed as fixed-point computation," *Information Processing Letters*, vol. 77, nos. 2–4, pp. 197–200, 2001. doi:10.1016/S0020-0190(00)00202-7.

[12] A. Lew, "Canonical greedy algorithms and dynamic programming," *Control and Cybernetics*, vol. 35, no. 3, pp. 621–643, 2006.